Testing

Membri del gruppo (Gruppo-03):

- Francesco Lemmo - 0612705508 (Group leader)
- Leonardo Luongo - 0612706039
- Francesco Monda - 0612705277
- Mattia Vitali - 0612706221

Planning test

La fase di testing è iniziata dopo la realizzazione di una prima versione funzionante (da linea di comando) del sistema. A questo punto dell'implementazione le unità da testare erano già definite, era ancora da sviluppare l'interfaccia grafica. A causa di questa situazione il carico di lavoro era da dividere tra la creazione dell'interfaccia grafica e degli Unit test. Le risorse sono state divise per portare a termine queste due fasi: Francesco Lemmo e Mattia Vitali si sono occupati dei testing, con l'ausilio di *Junit*; Leonardo Luongo e Francesco Monda hanno sviluppato la parte grafica, mediante l'utilizzo di JavaFX associato a SceneBuilder. Tutte e due le fasi sono state supervisionate da ogni membro.

Functional Test Cases

FTC-1	*Operazioni aritmetiche*
Preconditions	Calcolatrice in funzione e lo stack è vuoto
Flow of Events	Esegui "sqrt" -> (Elementi insufficenti nello stack: stack vuoto) Inserimento: 10 + 5j (visualizzazione nello stack) Inserimento: 4 – 2j (visualizzazione nello stack) Esegui "+" (visualizzazione nello stcak del risultato : 14 – 3j) Esegui "/" -> (Elementi insufficenti nello stack) Inserimento: 8957 Esegui "-" (visualizzazione nello stack del risultato: 8943 – 3j) Esegui "+-" (visualizzazione nello stack del risultato: -8943 + 3j) Chiudi la finestra.
Postconditions	Il programma è terminato
Use Cases	Inserisci numero – Numero Complesso – Operazione Aritmetica – Svolgi Operazione – Termina Programma

FTC-2	*Operazioni sullo stack*
Preconditions	Calcolatrice in funzione e lo stack è vuoto
Flow of Events	Inserimento: 18 Esesui "swap" -> (elementi insufficenti nello stack) Inserimento: 96 + 5j Esegui "dup" (visualizzazione della copia nello stack di: 96 + 5j) Esegui "drop" (cancellazione di 96 + 5j) Esegui "drop" (cancellazione di 96 + 5j) Esegui "over" -> (elementi insufficenti nello stack) Esegui "clear" (stack svuotato) Chiudi la finestra
Postconditions	Il programma è terminato
Use Cases	Inserisci numero – Numero Complesso – Operazione sullo stack – Svolgi Operazione – Termina Programma

FTC-3	*Operazioni con le variabili*
Preconditions	Calcolatrice in funzione e lo stack è vuoto
Flow of Events	Inserimento: 2 - j Esegui "<a" -> (la variabile è vuota) Inserimento: 4 – 7j Esegui ">a" (viene salvato nella variabile "a" il valore: 4 – 7j) Esegui "+a" (viene sommato il valore 2 -j con il contenuto di "a", la somma : 1 + 18j è salvata in "a") Esegui "<a" (viene inserito nello stack il valore: 1 + 18j) Chiudi la finestra
Postconditions	Il programma è terminato
Use Cases	Inserisci numero – Numero Complesso – Operazione con le variabili – Svolgi Operazione – Termina Programma

FTC-4	*Operazioni generiche*
Preconditions	Calcolatrice in funzione e lo stack è vuoto
Flow of Events	Inserimento: -36 Inserisci: 7 + 3j Esegui "swap" (vengono invertiti gli ultimi due elementi nello stack) Esegui "sqrt" (visualizzazione nello stack del risultato : 6j) Esegui ">f" (viene salvato nella variabile "f" il valore: 6j) Esegui "+" -> (Elementi insufficenti nello stack) Chiudi la finestra
Postconditions	Il programma è terminato
Use Cases	Inserisci numero – Numero Complesso – Operazione con le variabili – Svolgi Operazione – Termina Programma

Unit test

In questa fase abbiamo testato le singole unità del nostro software.
Abbiamo utilizzato un approccio *Bottom-Up*, collaudando prima i moduli di più basso livello nella gerarchia prodotta dalla progettazione fino al livello più alto.
I risultati di riferimento, per i test delle operazioni aritmetiche su valori complessi, sono stati prodotti con l'ausilio di WolframAlpha e MatLab.
I test di unità sono in totale 104. Ogni test è stato utile per il riconoscimento di falle nel sistema e lo sviluppo di un sistema il più robusto possibile.

Unit test cases: ComplexNumber

UTC-1.1.1	*Test ComplexNumber.fromString*
Test items	Class **Input validator,** method **fromString**
Input	String value = "+1.0032"; String expResult = new ComplexNumber(1.00320).toString(); String result = ComplexNumber.fromString(value).toString();
Oracle	result == 1.00320
Report	success
UTC-1.1.2	*Test ComplexNumber.fromString*
Test items	Class **Input validator,** method **fromString**
Input	String value = "-5.324"; String expResult = new ComplexNumber(-5.324).toString(); String result = ComplexNumber.fromString(value).toString();
Oracle	result == -5.324
Report	success
UTC-1.1.3	*Test ComplexNumber.fromString*
Test items	Class **Input validator,** method **fromString**
Input	String value = "9.167j";

	String expResult = new ComplexNumber(0,9.167).toString(); String result = ComplexNumber.fromString(value).toString();
Oracle	result == 0 + 9.167j
Report	success
UTC-1.1.4	*Test ComplexNumber.fromString*
Test items	Class **Input validator,** method **fromString**
Input	String value = "-11.3j"; String expResult = new ComplexNumber(0,-11.3).toString(); String result = ComplexNumber.fromString(value).toString();
Oracle	result == 0 -11.3j
Report	success
UTC-1.1.5	*Test ComplexNumber.fromString*
Test items	Class **Input validator,** method **fromString**
Input	String value = "3.54+ 6.32j"; String expResult = new ComplexNumber(3.54,6.32).toString(); String result = ComplexNumber.fromString(value).toString();
Oracle	result == 3.54 + 6.32j
Report	success
UTC-1.1.6	*Test ComplexNumber.fromString*
Test items	Class **Input validator,** method **fromString**
Input	String value = "-47.78 + 342.0j"; String expResult = new ComplexNumber(-47.78,342).toString(); String result = ComplexNumber.fromString(value).toString();
Oracle	result == -47.78 + 342j
Report	success
UTC-1.1.7	*Test ComplexNumber.fromString*
Test items	Class **Input validator,** method **fromString**
Input	String value = "+89.2 + 65.0j"; String expResult = new ComplexNumber(89.2,65.0).toString(); String result = ComplexNumber.fromString(value).toString();
Oracle	result == 89.2 + 65.0j
Report	success
UTC-1.1.8	*Test ComplexNumber.fromString*
Test items	Class **Input validator,** method **fromString**
Input	String value = "-7.758 - 376j"; String expResult = new ComplexNumber(-7.758,-376).toString(); String result = ComplexNumber.fromString(value).toString();
Oracle	result == -7.758 -376j
Report	success

Unit test cases: InputValidator

UTC-2.1.1	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("+12");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: +12)

Oracle	result == true
Report	success
UTC-2.1.2	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("-3");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: -3)

Oracle	result == true
Report	Success

UTC-2.1.3	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("+76.3");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: +76.4)

Oracle	result == true
Report	Success
UTC-2.1.4	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("-test");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: -test)

Oracle	result == false
Report	success
UTC-2.1.5	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("2 2");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: 2)

Oracle	throws IllegalArgumentException
Report	success
UTC-2.1.6	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("2j 2");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: 2j 2)

Oracle	result == false
Report	success
UTC-2.1.7	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("+2 +34 -3j");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: +2 +34 -3j)

Oracle	result == false
Report	success
UTC-2.1.8	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("*");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: *)

Oracle	result == true
Report	success
UTC-2.1.9	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("/");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: /)

Oracle	result == true
Report	success
UTC-2.1.10	*Test InputValidator.isValidInput*

Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("cia/o");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: cia/o)

Oracle	result == false
Report	success
UTC-2.1.11	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("c<");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: c<)

Oracle	result == false
Report	success
UTC-2.1.12	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("inv");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: inv)

Oracle	result == true
Report	success
UTC-2.1.13	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("dup");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: dup)

Oracle	result == true
Report	success
UTC-2.1.14	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("dUp");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: dUp)

Oracle	result == true
Report	success
UTC-2.1.15	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("5j");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: 5j)

Oracle	result == true
Report	success
UTC-2.1.16	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("""");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: “”)

Oracle	throws IllegalArgumentException
Report	success
UTC-2.1.17	*Test InputValidator.isValidInput*
Test items	Class **Input validator,** method **isValidInput**

Input	CorrectInputFormat correctInput = new CorrectInputFormat("null");
	boolean expResult = true;
	boolean result = instance.isValidInput(correctInput);
	(user input: "null")

Oracle	result == false
Report	success

UTC-2.2.1	*Test InputValidator.isComplexNumber*
Test items	Class **Input validator,** method **isComplexNumber**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("+48.3"); CorrectInputFormat("+48.3"); boolean result = instance.isComplexNumber(correctInput); **(user input: +48.3)**
Oracle	result == true
Report	success
UTC-2.2.2	*Test InputValidator.isComplexNumber*
Test items	Class **Input validator,** method **isComplexNumber**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("-73.11"); boolean result = instance.isComplexNumber(correctInput); **(user input:-73.11)**
Oracle	result == true
Report	success
UTC-2.2.3	*Test InputValidator.isComplexNumber*
Test items	Class **Input validator,** method **isComplexNumber**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("+76.4j"); boolean result = instance.isComplexNumber(correctInput); **(user input: +76.4j)**
Oracle	result == true
Report	success
UTC-2.2.4	*Test InputValidator.isComplexNumber*
Test items	Class **Input validator,** method **isComplexNumber**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("-234.222j"); boolean result = instance.isComplexNumber(correctInput); **(user input: -234.222j)**
Oracle	result == true
Report	success
UTC-2.2.5	*Test InputValidator.isComplexNumber*
Test items	Class **Input validator,** method **isComplexNumber**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("+2.23 + 34j"); boolean result = instance.isComplexNumber(correctInput); **(user input: +2.23 + 34j)**
Oracle	result == true
Report	success
UTC-2.2.6	*Test InputValidator.isComplexNumber*
Test items	Class **Input validator,** method **isComplexNumber**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("-65.4 + 863j"); boolean result = instance.isComplexNumber(correctInput); **(user input: -65.4 + 863j)**
Oracle	result == true
Report	success
UTC-2.2.7	*Test InputValidator.isComplexNumber*
Test items	Class **Input validator,** method **isComplexNumber**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("431 - 126.55j"); boolean result = instance.isComplexNumber(correctInput); **(user input: 431 – 126.55j)**
Oracle	result == true
Report	success

UTC-2.2.8	*Test InputValidator.isComplexNumber*
Test items	Class **Input validator,** method **isComplexNumber**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("-102.428 - 234j"); boolean result = instance.isComplexNumber(correctInput); **(user input: -102.428 – 234.j)**
Oracle	result == true
Report	success

UTC-2.3.1	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("+"); int result = instance.operationCode(correctInput); **(user input: +)**
Oracle	result == 0
Report	success
UTC-2.3.2	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("-"); int result = instance.operationCode(correctInput); **(user input: -)**
Oracle	result == 1
Report	success
UTC-2.3.3	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("*"); int result = instance.operationCode(correctInput); **(user input: *)**
Oracle	result == 2
Report	success
UTC-2.3.4	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("/"); int result = instance.operationCode(correctInput); **(user input: /)**
Oracle	result == 3
Report	success
UTC-2.3.5	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("sqrt"); int result = instance.operationCode(correctInput); **(user input: sqrt)**
Oracle	result == 4
Report	success
UTC-2.3.6	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("inv"); int result = instance.operationCode(correctInput); **(user input: inv)**
Oracle	result == 5
Report	success
UTC-2.3.7	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat(">v"); int result = instance.operationCode(correctInput); **(user input: >x)**
Oracle	result == 6

Report	success
UTC-2.3.8	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("<a"); int result = instance.operationCode(correctInput); **(user input: <x)**
Oracle	result == 7
Report	Success
UTC-2.3.9	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("+f"); int result = instance.operationCode(correctInput); **(user input: +x)**
Oracle	result == 8
Report	success
UTC-2.3.10	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("-g"); int result = instance.operationCode(correctInput); **(user input: -x)**
Oracle	result == 9
Report	success
UTC-2.3.11	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("dup"); int result = instance.operationCode(correctInput); **(user input: dup)**
Oracle	result == 10
Report	success
UTC-2.3.12	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("drop"); int result = instance.operationCode(correctInput); **(user input: drop)**
Oracle	result == 11
Report	success
UTC-2.3.13	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("swap"); int result = instance.operationCode(correctInput); **(user input: swap)**
Oracle	result == 12
Report	success
UTC-2.3.14	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("clear"); int result = instance.operationCode(correctInput); **(user input: clear)**
Oracle	result == 13
Report	success
UTC-2.3.15	*Test InputValidator.operationCode*
Test items	Class **Input validator,** method **operationCode**
Input	InputValidator.CorrectInputFormat correctInput = new CorrectInputFormat("over"); int result = instance.operationCode(correctInput); **(user input: over)**
Oracle	result == 14
Report	success

Unit test cases: Stack manager

UTC-3	*Test StackManager.setUp*

@BeforeEach
input
	sLast = new ComplexNumber(23,72); //second-last element
	last = new ComplexNumber(5,23.2); //last element
	instance.push(sLast);
	instance.push(last);

UTC-3.1.1	*Test StackManager.dup*
Test items	Class **Stack manager** , method **dup**
Input	instance.over(); int result = instance.top.toString();
Oracle	result == last
Report	success
UTC-3.1.2	*Test StackManager.dup*
Test items	Class **Stack manager** , method **dup**

Input	instance.pop();
	instance.pop();
	int result = instance.top.toString();

Oracle	throws NotEnoughElementsException
Report	success
UTC-3.2.1	*Test StackManager.drop*
Test items	Class **Stack manager** , method **drop**
Input	instance.drop(); int result = instance.top.toString();
Oracle	result == sLast
Report	success
UTC-3.2.2	*Test StackManager.drop*
Test items	Class **Stack manager** , method **drop**

Input	instance.pop();
	instance.pop();
	int result = instance.top.toString();

Oracle	throws NotEnoughElementsException
Report	success

UTC-3.3.1	*Test StackManager.swap*
Test items	Class **Stack manager** , method **swap**
Input	instance.swap(); int result = instance.top.toString();
Oracle	result == sLast
Report	success
UTC-3.3.2	*Test StackManager.swap*
Test items	Class **Stack manager** , method **swap**
Input	instance.pop(); int result = instance.top.toString();
Oracle	throws NotEnoughElementsException
Report	success

UTC-3.4.1	*Test StackManager.over*
Test items	Class **Stack manager** , method **over**
Input	instance.over(); int result = instance.top.toString();
Oracle	result == sLast
Report	success
UTC-3.4.2	*Test StackManager.over*
Test items	Class **Stack manager** , method **over**

Input	instance.pop(); int result = instance.top.toString();
Oracle	throws NotEnoughElementsException
Report	success

UTC-3.5.1	*Test StackManager.clear*
Test items	Class **Stack manager** , method **clear**
Input	instance.clear(); int result = instance.top.toString();
Oracle	result == last
Report	success

UTC-3.6.1	*Test StackManager.push*
Test items	Class **Stack manager** , method **push**

Input	ComplexNumber number = new ComplexNumber(51.3,24);
	instance.push(number);
	int result = instance.top.toString;

Oracle	result == number
Report	success

UTC-3.7.1	*Test StackManager.pop*
Test items	Class **Stack manager** , method **pop**
Input	ComplexNumber result = instance.pop();
Oracle	result == last
Report	success

UTC-3.8.1	*Test StackManager.top*
Test items	Class **Stack manager** , method **top**
Input	ComplexNumber result = instance.top();
Oracle	result == last
Report	success

Unit test cases: Calculator

UTC-4.1.1	*Test Calculator.add*
Test items	**Class Calcolatrice** , method **add**

Input	ComplexNumber cnExpected = new ComplexNumber(5.2, (2-0.56));
	calculator.fetchInput("2 + 2j");
	calculator.fetchInput("3.2 - 0.56j");
	calculator.fetchInput("+");
	result = calculator.getLastResult();

Oracle	result == 5.2 + 1.44j
Report	success
UTC-4.1.2	*Test Calculator.add*
Test items	**Class Calcolatrice** , method **add**

Input	ComplexNumber cnExpected = new ComplexNumber(0.0, -2+0.56);
	calculator.fetchInput("-2j");
	calculator.fetchInput("0.56j");
	calculator.fetchInput("+");
	result = calculator.getLastResult();

Oracle	result == 0.0 - 1.44j
Report	success
UTC-4.1.3	*Test Calculator.add*
Test items	**Class Calcolatrice** , method **add**

Input	ComplexNumber cnExpected = new ComplexNumber(3);
	calculator.fetchInput("-2.46");
	calculator.fetchInput("5.46");

	calculator.fetchInput("+"); result = calculator.getLastResult();
Oracle	result == 3
Report	success
UTC-4.1.4	*Test Calculator.add*
Test items	**Class Calcolatrice** , method **add**

Input	ComplexNumber cnExpected = new ComplexNumber(505, 104);
	calculator.fetchInput("505");
	calculator.fetchInput("104j");
	calculator.fetchInput("+");
	result = calculator.getLastResult();

Oracle	result == 505 + 104j
Report	success
UTC-4.1.5	*Test Calculator.add*
Test items	**Class Calcolatrice** , method **add**

Input	ComplexNumber cnExpected = new ComplexNumber(0, 0);
	calculator.fetchInput("0");
	calculator.fetchInput("0j");
	calculator.fetchInput("+")
	result = calculator.getLastResult();

Oracle	result == 0
Report	success
UTC-4.1.6	*Test Calculator.add*
Test items	**Class Calcolatrice** , method **add**

Input	ComplexNumber cnExpected = new ComplexNumber(0, 0);
	calculator.fetchInput("5+8j");
	calculator.fetchInput("-5-8j");
	calculator.fetchInput("+");
	result = calculator.getLastResult();

Oracle	result == 0
Report	success
UTC-4.1.7	*Test Calculator.add*
Test items	**Class Calcolatrice** , method **add**
Input	calculator.fetchInput("64+8j"); calculator.fetchInput("+");
Oracle	throws NotEnoughElementsException
Report	success

UTC-4.2.1	*Test Calculator.div*
Test items	**Class Calcolatrice** , method **div**

Input	ComplexNumber cnExpected = new ComplexNumber(0.8, 0.6);
	calculator.fetchInput("1+2j");
	calculator.fetchInput("2+1j");
	calculator.fetchInput("/");
	result = calculator.getLastResult();

Oracle	result == 0.8 + 0.6j
Report	success
UTC-4.2.2	*Test Calculator.div*
Test items	**Class Calcolatrice** , method **div**

Input	ComplexNumber cnExpected = new ComplexNumber(-2.0, 54.0);
	calculator.fetchInput("54 + 2j");
	calculator.fetchInput("0 - 1j");
	calculator.fetchInput("/");
	result = calculator.getLastResult();

Oracle	result == -2.0 + 54.0j
Report	success
UTC-4.2.3	*Test Calculator.div*

Test items	**Class Calcolatrice** , method **div**

Input	ComplexNumber cnExpected = new ComplexNumber(0.1083, 0.4675);
	calculator.fetchInput("0.32 - 5j");
	calculator.fetchInput("-10 - 3j");
	calculator.fetchInput("/");
	result = calculator.getLastResult();

Oracle	result == 0.1083 + 0.4675j
Report	success
UTC-4.2.4	*Test Calculator.div*
Test items	**Class Calcolatrice** , method **div**

Input	ComplexNumber cnExpected = new ComplexNumber(0.2829, 2.0259);
	calculator.fetchInput("198.09 - 13.2j");
	calculator.fetchInput("7 - 96.801j");
	calculator.fetchInput("/");
	result = calculator.getLastResult();

Oracle	result == 0.2829 + 2.0259j
Report	success
UTC-4.2.5	*Test Calculator.div*
Test items	**Class Calcolatrice** , method **div**

Input	ComplexNumber cnExpected = new ComplexNumber(0, 0);
	calculator.fetchInput("0");
	calculator.fetchInput("1j");
	calculator.fetchInput("/");
	result = calculator.getLastResult();

Oracle	result == 0
Report	success
UTC-4.2.6	*Test Calculator.div*
Test items	**Class Calcolatrice** , method **div**

Input	ComplexNumber cnExpected = new ComplexNumber(-1, 0.0);
	calculator.fetchInput("5+8j");
	calculator.fetchInput("-5-8j");
	calculator.fetchInput("/");
	result = calculator.getLastResult();

Oracle	result == -1
Report	success
UTC-4.2.7	*Test Calculator.div*
Test items	**Class Calcolatrice** , method **div**

Input	calculator.fetchInput("1");
	calculator.fetchInput("0");
	calculator.fetchInput("/");

Oracle	Throws ArithmeticExceptions
Report	success

UTC-4.3.1	*Test Calculator.mul*
Test items	**Class Calcolatrice** , method **mul**

Input	ComplexNumber cnExpected = new ComplexNumber(0, 29.0);
	calculator.fetchInput("5 + 2j");
	calculator.fetchInput("2 + 5j");
	calculator.fetchInput("*");
	result = calculator.getLastResult();

Oracle	result == 29.0j
Report	success
UTC-4.3.2	*Test Calculator.mul*
Test items	**Class Calcolatrice** , method **mul**

Input	ComplexNumber cnExpected = new ComplexNumber(2.0, -54.0);
	calculator.fetchInput("54 + 2j");
	calculator.fetchInput("0 - 1j");

	calculator.fetchInput("*"); result = calculator.getLastResult();
Oracle	result == 2.0 - 54.0j
Report	success
UTC-4.3.3	*Test Calculator.mul*
Test items	**Class Calcolatrice** , method **mul**

Input	ComplexNumber cnExpected = new ComplexNumber(-18.2, 49.04);
	calculator.fetchInput("0.32 - 5j");
	calculator.fetchInput("-10 - 3j");
	calculator.fetchInput("*");
	result = calculator.getLastResult();

Oracle	result == -18.2 + 49.04j
Report	success
UTC-4.3.4	*Test Calculator.mul*
Test items	**Class Calcolatrice** , method **mul**

Input	ComplexNumber cnExpected = new ComplexNumber(108.86, -19268);
	calculator.fetchInput("198.09 - 13.2");
	calculator.fetchInput("7 - 96.801j");
	calculator.fetchInput("*");
	result = calculator.getLastResult();

Oracle	result == 108.86 - 19268j
Report	success
UTC-4.3.5	*Test Calculator.mul*
Test items	**Class Calcolatrice** , method **mul**

Input	ComplexNumber cnExpected = new ComplexNumber(0, 0);
	calculator.fetchInput("0");
	calculator.fetchInput("0j");
	calculator.fetchInput("*");
	result = calculator.getLastResult();

Oracle	result == 0
Report	success
UTC-4.3.6	*Test Calculator.mul*
Test items	**Class Calcolatrice** , method **mul**

Input	ComplexNumber cnExpected = new ComplexNumber(39, -80);
	calculator.fetchInput("5+8j");
	calculator.fetchInput("-5-8j");
	calculator.fetchInput("*");
	result = calculator.getLastResult();

Oracle	result == 39 - 80j
Report	success
UTC-4.3.7	*Test Calculator.mul*
Test items	**Class Calcolatrice** , method **mul**
Input	calculator.fetchInput("64+8j"); calculator.fetchInput("*");
Oracle	throws NotEnoughElementsException
Report	success

UTC-4.4.1	*Test Calculator.sub*
Test items	**Class Calcolatrice** , method **sub**

Input	ComplexNumber cnExpected = new ComplexNumber(- 21 + 90, 34 + 5.6);
	calculator.fetchInput("21 + 34j");
	calculator.fetchInput("90 - 5.6j");
	calculator.fetchInput("-");
	result = calculator.getLastResult();

Oracle	result == -69 + 39.6j
Report	success
UTC-4.4.2	*Test Calculator.sub*

Test items	**Class Calcolatrice** , method **sub**

Input	ComplexNumber cnExpected = new ComplexNumber(0, -2-0.56);
	calculator.fetchInput("-2j");
	calculator.fetchInput("0.56j");
	calculator.fetchInput("-");
	result = calculator.getLastResult();

Oracle	result == -2.56j
Report	success
UTC-4.4.3	*Test Calculator.sub*
Test items	**Class Calcolatrice** , method **sub**

Input	ComplexNumber cnExpected = new ComplexNumber(-85-0.9546, 0.0);
	calculator.fetchInput("-85");
	calculator.fetchInput("0.9546");
	calculator.fetchInput("-");
	result = calculator.getLastResult();

Oracle	result == -85.9546
Report	success
UTC-4.4.4	*Test Calculator.sub*
Test items	**Class Calcolatrice** , method **sub**

Input	ComplexNumber cnExpected = new ComplexNumber(71, -43);
	calculator.fetchInput("71");
	calculator.fetchInput("43j");
	calculator.fetchInput("-");
	result = calculator.getLastResult();

Oracle	result == 71 - 43j
Report	success
UTC-4.4.5	*Test Calculator.sub*
Test items	**Class Calcolatrice** , method **sub**

Input	ComplexNumber cnExpected = new ComplexNumber(0, 0);
	calculator.fetchInput("0");
	calculator.fetchInput("0j");
	calculator.fetchInput("-");
	result = calculator.getLastResult();

Oracle	result == 0
Report	success
UTC-4.4.6	*Test Calculator.sub*
Test items	**Class Calcolatrice** , method **sub**

Input	ComplexNumber cnExpected = new ComplexNumber(10, 16);
	calculator.fetchInput("5+8j");
	calculator.fetchInput("-5-8j");
	calculator.fetchInput("-");
	result = calculator.getLastResult();

Oracle	result == 10 + 16j
Report	success

UTC-4.5.1	*Test Calculator.inv*
Test items	**Class Calcolatrice** , method **inv**

Input	ComplexNumber cnExpected = new ComplexNumber(-2, 1);
	calculator.fetchInput("2 – 1j");
	calculator.fetchInput("inv");
	result = calculator.getLastResult();

Oracle	result == -2+1j
Report	success
UTC-4.5.2	*Test Calculator.inv*
Test items	**Class Calcolatrice** , method **inv**
Input	ComplexNumber cnExpected = new ComplexNumber(43.189, -0.98j); calculator.fetchInput("-43.189 + 0.98j");

	calculator.fetchInput("inv"); result = calculator.getLastResult();
Oracle	result == 43.189-0.98j
Report	success
UTC-4.5.3	*Test Calculator.inv*
Test items	**Class Calcolatrice** , method **inv**

Input	ComplexNumber cnExpected = new ComplexNumber(-9, 65.20);
	calculator.fetchInput("9 - 65.20j ");
	calculator.fetchInput("inv");
	result = calculator.getLastResult();

Oracle	result == -9 + 65.20j
Report	success
UTC-4.5.4	*Test Calculator.inv*
Test items	**Class Calcolatrice** , method **inv**

Input	ComplexNumber cnExpected = new ComplexNumber(0, -12.30);
	calculator.fetchInput("12.30j");
	calculator.fetchInput("inv");
	result = calculator.getLastResult();

Oracle	result == 0-12.30j
Report	success
UTC-4.5.5	*Test Calculator.inv*
Test items	**Class Calcolatrice** , method **inv**

Input	ComplexNumber cnExpected = new ComplexNumber(-10023, 0);
	calculator.fetchInput("10023");
	calculator.fetchInput("inv");
	result = calculator.getLastResult();

Oracle	result == -10023+0j
Report	success
UTC-4.5.6	*Test Calculator.inv*
Test items	**Class Calcolatrice** , method **inv**

Input	ComplexNumber cnExpected = new ComplexNumber(0, 0);
	calculator.fetchInput("0");
	calculator.fetchInput("inv");
	result = calculator.getLastResult();

Oracle	result == 0+0j
Report	success
UTC-4.5.7	*Test Calculator.inv*
Test items	**Class Calcolatrice** , method **inv**

Input	ComplexNumber cnExpected = new ComplexNumber(-897.1, -9j);
	calculator.fetchInput("897.1 + 9j");
	calculator.fetchInput("inv");
	result = calculator.getLastResult();

Oracle	result == -897.1-9j
Report	success

UTC-4.6.1	*Test Calculator.sqrt*
Test items	**Class Calcolatrice** , method **sqrt**

Input	ComplexNumber cnExpected = new ComplexNumber(0, 2);
	calculator.fetchInput("-4");
	calculator.fetchInput("sqrt");
	result = calculator.getLastResult();

Oracle	result == 0 + 2j
Report	success
UTC-4.6.2	*Test Calculator.sqrt*
Test items	**Class Calcolatrice** , method **sqrt**
Input	ComplexNumber cnExpected = new ComplexNumber(2.59390, 1.19511); calculator.fetchInput("5.3+6.2j");

	calculator.fetchInput("sqrt"); result = calculator.getLastResult();
Oracle	result == 2.59390 + 1.19511j
Report	success
UTC-4.6.3	*Test Calculator.sqrt*
Test items	**Class Calcolatrice** , method **sqrt**

Input	ComplexNumber cnExpected = new ComplexNumber(2.44948, 2.44948);
	calculator.fetchInput("12j");
	calculator.fetchInput("sqrt");
	result = calculator.getLastResult();

Oracle	result == 2.44948 + 2.44948j
Report	success

UTC-4.7.1	*Test Calculator.varPlusStack*
Test items	**Class Calcolatrice** , method **varPlusStack**

Input	calculator.fetchInput("4");
	String var = "v";
	calculator.fetchInput("5");
	calculator.fetchInput(">"+var);
	calculator.fetchInput("+"+var);
	ComplexNumber expResult = new ComplexNumber(9);

Oracle	result == 9
Report	success
UTC-4.7.2	*Test Calculator.varPlusStack*
Test items	**Class Calcolatrice** , method **varPlusStack**

Input	calculator.fetchInput("-21+3.12j");
	String var = "v";
	calculator.fetchInput("35.2 + 53j");
	calculator.fetchInput(">"+var);
	calculator.fetchInput("+"+var);
	ComplexNumber expResult = new ComplexNumber(14.2, 56.12);

Oracle	result == 14.2 + 56.12j
Report	success
UTC-4.7.3	*Test Calculator.varPlusStack*
Test items	**Class Calcolatrice** , method **varPlusStack**

Input	calculator.fetchInput("4.43j");
	String var = "v";
	calculator.fetchInput("25");
	calculator.fetchInput(">"+var);
	calculator.fetchInput("+"+var);
	ComplexNumber expResult = new ComplexNumber(25,4.43);

Oracle	result == 25 + 4.43j
Report	success

UTC-4.8.1	*Test Calculator.varSubStack*
Test items	**Class Calcolatrice** , method **varSubStack**

Input	calculator.fetchInput("4+3j");
	String var = "v";
	calculator.fetchInput("7+12j");
	calculator.fetchInput(">"+var);
	calculator.fetchInput("-"+var);
	ComplexNumber expResult = new ComplexNumber(3,9);

Oracle	result == 3,9
Report	success
UTC-4.8.2	*Test Calculator.varSubStack*
Test items	**Class Calcolatrice** , method **varSubStack**
Input	calculator.fetchInput("-21+21j");

	String var = "v";
	calculator.fetchInput("35.2 + 53j");
	calculator.fetchInput(">"+var);
	calculator.fetchInput("-"+var);
	ComplexNumber expResult = new ComplexNumber(56.2,32);

Oracle	result == 56.2 + 32j
Report	success
UTC-4.8.3	*Test Calculator.varSubStack*
Test items	**Class Calcolatrice** , method **varSubStack**

Input	calculator.fetchInput("7.43j");
	String var = "v";
	calculator.fetchInput("5.32");
	calculator.fetchInput(">"+var);
	calculator.fetchInput("-"+var);
	ComplexNumber expResult = new ComplexNumber(5.32, -7.43);

Oracle	result == 5.32 - 7.43j
Report	success

UTC-4.9.1	*Test Calculator.fromStackIntoVariable*
Test items	**Class Calcolatrice** , method **fromStackIntoVariable**

Input	ComplexNumber expResult = new ComplexNumber(4.3,9.12);
	calculator.fetchInput("4.3 + 9.12j");
	String var = "a";
	calculator.fetchInput(">"+var);
	ComplexNumber result = calculator.variableManager.getVariable(var).getValue();

Oracle	result == 4.3 + 9.12j
Report	success
UTC-4.9.2	*Test Calculator.fromStackIntoVariable*
Test items	**Class Calcolatrice** , method **fromStackIntoVariable**
Input	calculator.fetchInput(">x");
Oracle	throws RunTimeExceptions
Report	success

UTC-4.10.1	*Test Calculator.fromVariableIntoStack*
Test items	**Class Calcolatrice** , method **fromVariableIntoStack**

Input	ComplexNumber expResult = new ComplexNumber(8.33,15.2);
	calculator.fetchInput("8.33 + 15.2j");
	String var = "a";
	calculator.fetchInput(">"+var);
	calculator.fetchInput("<"+var);
	ComplexNumber result = getLastResult();

Oracle	result == 8.33 + 15.2j
Report	success
UTC-4.10.2	*Test Calculator.fromVariableIntoStack*
Test items	**Class Calcolatrice** , method **fromVariableIntoStack**
Input	calculator.fetchInput("<x");
Oracle	throws RunTimeExceptions
Report	success

Failure Tests

Per mezzo dei test eseguiti e grazie alla loro quantità, tutti i casi di fallimento notati durante lo sviluppo del sistema sono stati risolti con successo.
Una problematica minore che abbiamo riscontrato è relativa all'aspetto dell'interfaccia grafica. Quando il sistema viene eseguito su dispositivi diversi, infatti, potrebbe modificarsi l'aspetto di acuni bottoni. Questo problema non influisce sulla funzionalità.

Tabella di Tracciabilità

La tabella di tracciabilità è stata aggiornata riportando in essa i test delle relative unità del sistema.

#	Requirement	Design	Implementation	Testing
1.0	**Stack data structure**	StackManager	StackManager.java	UTC-3
2.0	**Input number(s) or input operation**	Calculator	Calculator.java	UTC-2
3.0	**Text Area**	UserInterface	Calculator_v2.fxml	/
4.0	**Complex numbers**	ComplexNumber	ComplexNumber.java	UTC-1
5.0	**Operazioni aritmetiche:**	Calculator	Calculator.java	UTC-4
5.1	**Addizione**	Calculator	Calculator.java	UTC-4.1
5.2	**Differenza**	Calculator	Calculator.java	UTC-4.4
5.3	**Moltiplicazione**	Calculator	Calculator.java	UTC-4.3
5.4	**Divisione**	Calculator	Calculator.java	UTC-4.2
5.5	**Radice quadrata**	Calculator	Calculator.java	UTC-4.6
5.6	**Inversione del segno**	Calculator	Calculator.java	UTC-4.5
6.0	**Flusso di esecuzione delle operazioni**	Calculator	Calculator.java	/
7.0	**Operazioni sullo stack:**	StackManager	StackManager.java	UTC-3
7.1	**CLEAR**	StackManager	StackManager.java	UTC-3.5
7.2	**DROP**	StackManager	StackManager.java	UTC-3.2
7.3	**DUP**	StackManager	StackManager.java	UTC-3.1
7.4	**SWAP**	StackManager	StackManager.java	UTC-3.3
7.5	**OVER**	StackManager	StackManager.java	UTC-3.4
8.0	**26 variabili**	VariablesManager	VariableManager.java	/
9.0	**Operazioni sulle variabili:**	Calculator	Calculator.java	UTC-4
9.1	**Salva nella variabile (>x)**	Calculator	Calculator.java	UTC-4.9
9.2	**Copia nello stack (<x)**	Calculator	Calculator.java	UTC-4.10
9.3	**Addizione (+x)**	Calculator	Calculator.java	UTC-4.7
9.4	**Sottrazione (-x)**	Calculator	Calculator.java	UTC-4.8
10.0	**Intefaccia utente**	UserInterface	Calculator_v2.fxml	/